

BoxBreaker — Persistencia de archivos

Capa de acceso a datos | qué cubre hoy y qué falta para persistencia RPG completa | 20 de junio de 2026

Resumen

La capa de acceso a datos es una **versión inicial funcional**, no persistencia RPG completa. Un archivo declarado **es** una tabla física real (con RRN) y el ciclo CRUD por RRN (WRITE/READ/CHAIN/UPDATE/DELETE) anda de punta a punta, más SQL interactivo (STRSQL). Es un esqueleto vertical que prueba el modelo. Pero le falta lo que más define a un archivo de IBM i: **acceso por clave**, el **modelo stateful de open/cursor**, **bloqueo de registro** y **commitment control** — más una lista de items incrementales (tipos y reglas de DDS, constraints, marshaling type-fiel). En una frase: tenemos el **almacenamiento + el read/write crudo por RRN**; falta el **método de acceso** que RPG realmente consume.

Almacenamiento + CRUD FUNCIONAL PF→tabla · RRN CRUD · STRSQL	Acceso por clave FALTA CHAIN/SETLL/READE · LF	Open/cursor + locking FALTA ODP por job · commit	Tipos / constraints PARCIAL 4 tipos · sin reglas DDS
--	--	---	---

1 Lo que ya cubre (verificado end-to-end)

Capacidad	Estado	Detalle
Declarar archivo (CRTPF)	✓ Hecho	El PF declarado crea una tabla física real con RRN (clave) + las columnas del record format
Cambiar estructura (CHGPF)	✓ Hecho	ALTER TABLE: DROP/ADD/MODIFY de columnas, en un único ALTER atómico
WRITE / READ / CHAIN	✓ Hecho	INSERT (devuelve RRN), SELECT secuencial (orden RRN), lectura aleatoria por RRN
UPDATE / DELETE	✓ Hecho	Por RRN; el UPDATE parcial respeta los campos no enviados
SQL interactivo (STRSQL)	✓ Hecho	Cualquier sentencia directa contra la base; result set o filas afectadas
Tipos de campo	✓ Hecho	CHAR, DECIMAL (exacto), INTEGER, DATE → columna SQL
Seguridad del acceso	✓ Hecho	Sólo columnas del record format; valores siempre como parámetros JDBC (sin inyección)

Es una base real y funcional: el ciclo **declarar** → **ver/cambiar estructura** → **consultar (SQL)** → **leer/escribir registros** está cerrado y probado. Lo que sigue es lo que separa esto de "persistencia RPG".

2 Falta — fundamental (cambia el modelo, no es un agregado)

Lo que falta	Impacto	Por qué importa
Acceso por clave / access paths	● Alto	El RPG real casi no usa RRN: usa claves. CHAIN por campo clave, SETLL/SETGT/READE/READP que posicionan y leen en orden de clave. Hoy sólo hay CHAIN por RRN. Sin esto no es acceso RPG
Logical files (LF)	● Alto	Órdenes de acceso alternativos, selección/omisión de registros y join logical files sobre los physical files
Modelo stateful open/cursor (ODP)	● Alto	OPEN/CLOSE con posición actual por job: READ avanza, READE lee el siguiente-igual. Hoy las operaciones son sin estado. Es, además, la ABI stateful que diseñamos para v2
Bloqueo de registro (locking)	● Alto	El UPDATE de RPG lee con lock y libera al actualizar. Hoy no hay locks → updates concurrentes harían carrera
Commitment control / transacciones	● Alto	COMMIT/ROLBK y journaling. Hoy cada operación auto-commitea; no hay límites transaccionales multi-operación

3 Falta — incremental (importante, pero se suma de a poco)

Lo que falta	Impacto	Por qué importa
Tipos y reglas de DDS	● Medio	Packed vs zoned, varchar, time/timestamp con formatos, campos null-capable (ALWNULL) y valores por defecto (DFT)
Constraints	● Medio	Claves UNIQUE, clave primaria de negocio, foreign keys, validación CHECK/RANGE/VALUES/COMP
Marshaling type-fiel	● Medio	Los decimales entran como Double (riesgo de precisión float); deberían viajar como BigDecimal. Hoy lo tapa que MySQL redondea a la escala
DDL transaccional	● Medio	CREATE/ALTER hacen commit implícito en MySQL → posible tabla huérfana ante un fallo raro (mitigado validando antes)
Presentación de campos	■ Bajo	Edit codes (EDTCDE/EDTWRD), column headings, campos por referencia (REFFLD)

4 Falta — borde / más adelante

Lo que falta	Impacto	Por qué importa
Members	■ Bajo	Un PF puede tener varios members; OVRDBF para redirigir archivo/member en runtime
Multi-format / join	■ Bajo	Archivos con varios formatos de registro; join logical files de varios PF
Triggers e índices extra	■ Bajo	Triggers de base de datos e índices de performance más allá de la clave

5 Bottom line y próximo paso

Lo construido es el **disco + el read/write crudo por RRN**: sólido y verificado, pero es el primitivo de almacenamiento. Lo que lo convierte en "persistencia RPG" de verdad es la capa de arriba — el **método de**

acceso: acceso por clave, el modelo stateful de open/cursor, locking y transacciones. Esos cuatro (Impacto alto) son los que más definen "esto es un archivo de IBM i".

Próximo paso de mayor impacto: el **acceso por clave** (definir campos clave en el archivo + CHAIN/SETLL/READE por clave) — es lo que desbloquea correr programas RPG reales sobre estos archivos. El modelo stateful open/cursor va de la mano y es, además, la ABI de datos de la versión v2 (programa corriendo adentro del runtime).

Generado a partir del estado del repositorio. Impacto: ● **Alto** ● **Medio** ■ **Bajo**.